Comprehensive Technical Report: Stereo Matching Pipeline System Architecture

Blaze

October 24, 2025

Abstract

This technical document provides a comprehensive description of a stereo matching pipeline system architecture. The system implements a complete workflow from web-based data acquisition to trained neural network models for disparity estimation. The pipeline is organized into four modular components with well-defined interfaces and data flow. Each module handles specific responsibilities including data collection, preprocessing, feature extraction, and model training.

1 System Architecture Overview

The stereo matching pipeline is designed as a sequential processing system with four interconnected modules. The architecture follows a data-flow pattern where each module consumes the output of the previous module and produces input for the next.

2 Module 1: Data Acquisition System

2.1 Module Purpose and Scope

The data acquisition module is responsible for automatically retrieving the Middlebury 2014 stereo dataset from the official web repository. It handles web scraping, file discovery, directory structure generation, and robust download management.

2.2 Core Algorithm Implementation

Algorithm 1 Data Acquisition Main Workflow

```
procedure AcquireDataset(base_url, download_path)
    Initialize web session with browser headers
    ScanForZipFiles(base_url)                          ▷ Find dataset containers
    FilterDatasets(file_list, exclude_keywords)        ▷ Remove imperfect datasets
    GenerateSiteStructure(dataset_list)                ▷ Create directory hierarchy

    for each dataset in dataset_list do
        DownloadCoreFiles(dataset_url, download_path)
        DownloadExposureVariants(dataset_url, download_path)
    end for
    return VerifyDownloads(download_path)
end procedure
```

2.3 Key Component Details

2.3.1 Web Scraping Engine

Algorithm 2 File Discovery Algorithm

```
function FindDownloadables(url, extensions, filenames)
    response ← HTTP_GET(url) with custom headers
    soup ← BeautifulSoup(response.content)
    downloadables ← []

    for each anchor tag in soup do
        href ← extract href attribute

        if href is valid and not javascript then
            absolute_url ← urljoin(base_url, href)

            if url matches extensions or filenames then
                downloadables.append({
                    'filename': extract_filename(href),
                    'link': absolute_url
                })
            end if
        end if
    end for
    return downloadables
end function
```

2.3.2 Directory Structure Generator

Algorithm 3 Site Structure Generation

```
function GenerateOrgSite(zip_files)
    org_sites ← []

    for each zip_file in zip_files do
        base_path ← replace_extension(zip_file.link, ")
        dataset_path ← replace_substring(base_path, 'zip', 'datasets')
        org_sites.append(dataset_path + '/')

        for level in ['L1', 'L2', 'L3', 'L4'] do
            org_sites.append(dataset_path + '/ambient/' + level + '/')
        end for
    end for
    return org_sites
end function
```

2.3.3 File Download Manager

Algorithm 4 Robust File Download

```
function DownloadFiles(file_list, download_location, org_site)

    for each file_info in file_list do
        file_url ← file_info['link']
        relative_path ← extract_relative_path(file_url, org_site)
        full_path ← os.path.join(download_location, relative_path)

        if not file_exists(full_path) then
            create_directories(os.path.dirname(full_path))
            stream_download(file_url, full_path)
            log_success(file_info['filename'])
        else
            log_skip(file_info['filename'])
        end if
    end for
end function
```

2.4 Data Structures and Output

The module produces a organized directory structure containing:

- **Stereo Images**: `im0.png`, `im1.png` (left/right pairs)
- **Disparity Maps**: `disp0.pfm`, `disp1.pfm` (floating-point format)
- **Calibration Files**: `calib.txt` (camera parameters)

- **Exposure Variants**: Multiple illumination conditions per scene

3 Module 2: Data Preprocessing System

3.1 Module Purpose and Scope

Transforms raw downloaded data into standardized, cleaned formats suitable for machine learning. Handles file parsing, image processing, data augmentation, and serialization.

3.2 Core Processing Pipeline

Algorithm 5 Data Preprocessing Main Workflow

```
procedure PreprocessData(raw_path, output_path, resize_factor)
    calibration_data ← ProcessAllCalibrations(raw_path)
    image_data ← ProcessAllImages(raw_path, resize_factor)
    disparity_data ← ProcessAllDisparities(raw_path, resize_factor)
    SerializeData(calibration_data, image_data, disparity_data, output_path)
    return file_paths
end procedure
```

3.3 Component Algorithms

3.3.1 Calibration Parser

Algorithm 6 Calibration File Processing

```
function LoadCalibration(file_path)
    calib ← {}

    for each line in file do

        if line contains '=' then
            key, value ← split(line, '=', 1)

            if value starts with '[' and ends with ']' then
                matrix_str ← value[1:-1]
                rows ← split(matrix_str, ';')
                matrix ← []

                for each row in rows do
                    matrix.append([float(x) for x in split(row)])
                end for
                calib[key] ← np.array(matrix)
            else

                numeric_value ← float(value)

                if numeric_value is integer then
                    calib[key] ← int(numeric_value)
                else
                    calib[key] ← numeric_value
                end if
                calib[key] ← value.strip()
            end if
        end if
    end for
    return calib
end function
```

3.3.2 Disparity Map Processor

Algorithm 7 PFM Disparity Map Loading and Cleaning

```
function LoadPFMDisparity(file_path)
    Read header: magic_word, dimensions, scale
    width, height ← parse(dimensions)
    scale_factor ← parse(scale)
    little_endian ← (scale_factor < 0)

    if little_endian then
        data ← np.fromfile(file, dtype='¡f4')
    else
        data ← np.fromfile(file, dtype='¿f4')
    end if

    if len(data) = width × height then
        disparity ← reshape(data, (height, width))
    else if len(data) = width × height × 3 then
        disparity ← mean(reshape(data, (height, width, 3)), axis=2)
    else
        Raise ValueError
    end if
    disparity ← flipud(disparity)                          ▷ Vertical flip
    return CleanDisparity(disparity)
end function
```

3.3.3 Disparity Cleaning Algorithm

Algorithm 8 Disparity Map Cleaning and Normalization

```
function CleanArray(arr, max_disp)
    result ← where(arr < 0, 0, arr)                  ▷ Remove negative values
    finite_mask ← isfinite(result)

    if any(finite_mask) then
        max_val ← max(result[finite_mask])
    else
        max_val ← 0
    end if
    result ← where(isinf(result), max_val, result)          ▷ Replace infinity

    if max_val > max_disp then
        result ← (result / max_val) × max_disp                    ▷ Normalize
    end if
    return result.astype(np.int16)
end function
```

3.3.4 Image Processing Pipeline

Algorithm 9 Image Loading and Processing

```
function ProcessImage(image_path, resize_factor)
    rgb_array ← PIL.Image.open(image_path).convert('RGB')
    gray_array ← dot(rgb_array[...,:3], [0.33, 0.33, 0.33])

    if resize_factor ≠ 1.0 then
        gray_array ← ResizeImage(gray_array, resize_factor)
    end if
    return gray_array.astype(np.uint8)
end function
function ResizeImage(image_array, scale_factor)

    if image_array.ndim = 2 then                                  ▷ Grayscale
        img ← PIL.Image.fromarray(image_array)
    else                                                          ▷ Color
        img ← PIL.Image.fromarray(image_array.astype('uint8'))
    end if
    new_size ← (int(width × scale_factor), int(height × scale_factor))
    resized ← img.resize(new_size, PIL.Image.LANCZOS)
    return np.array(resized)
end function
```

3.4 Data Augmentation Strategy

Algorithm 10 Data Augmentation through Flipping

```
procedure GenerateAugmentedData(images, disparities)
    augmented_images ← {'org': [], 'flip': []}
    augmented_disparities ← {'org': [], 'flip': []}

    for each original image pair do
        AddOriginal(augmented_images, augmented_disparities)
        AddFlipped(augmented_images, augmented_disparities)
    end for
    return augmented_images, augmented_disparities
end procedure
function AddFlipped(images, disparities, original_idx)
    left_flipped ← fliplr(original_right)
    right_flipped ← fliplr(original_left)
    disp_left_flipped ← fliplr(original_disp_right)
    disp_right_flipped ← fliplr(original_disp_left)
    Store flipped pairs with original index
end function
```

4 Module 3: Patch Generation and Feature Extraction

4.1 Module Purpose and Scope

Generates training samples by extracting image patches and computing feature descriptors. Creates corresponding patch-strip pairs across the entire disparity range with feature-based validation.

4.2 Core Generation Algorithm

Algorithm 11 Patch Generation Main Workflow

```
procedure GeneratePatchesStrips(output_path, patch_shape, target_samples, params)
    Initialize memory-mapped arrays for all data types
    pbar ← tqdm(total=target_samples)

    for i = 0 to target_samples-1 do
        FindValidPatch(i, patch_shape, params)

        if found valid patch then
            ExtractPatchStripPair(i, patch_location, params)
            ComputeAndStoreFeatures(i)
        else
            HandleFailedPatch(i)
        end if

        if i mod flush_interval = 0 then
            FlushArrays
        end if
    end for
    FinalFlush
    return array_paths
end procedure
```

4.3 Patch Sampling Algorithm

Algorithm 12 Random Patch Sampling with Validation

```
function FindValidPatch(sample_index, patch_shape, params)
    tries ← 0

    while tries < max_tries and not found do
        side ← random.choice(['org', 'flip'])
        truth_data ← random.choice(truth_data[side])
        image_data ← random.choice(image_data[side][truth_data.index])
        SampleRandomLocation(image_data, patch_shape, params)
        truth_disp ← median(disp_patch)

        if min_disp ≤ truth_disp < max_disp then

            if ValidatePatch(image_patch, truth_disp, params) then
                return (patch_location, truth_disp, image_data)
            end if
        end if
        tries ← tries + 1
    end while
    return None                                          ▷ No valid patch found
end function
function SampleRandomLocation(image_data, patch_shape, params)
    img_height, img_width ← image_data.disparity.shape
    y_min ← random.randint(0, img_height - patch_shape[0])
    x_min ← random.randint(params.max_disp, img_width - patch_shape[1])
    y_max ← y_min + patch_shape[0]
    x_max ← x_min + patch_shape[1]
    return (y_min, y_max, x_min, x_max)
end function
```

4.4 Feature Extraction System

Algorithm 13 Comprehensive Feature Computation

function ComputeFeatures(patch, eps=1e-6)
    patch ← patch.astype(np.float32)
    ▷ 1. Statistical Moments
    mean_val ← mean(patch)
    std_val ← std(patch) / 0.5 ▷ Normalize to [0,1]
    skew_val ← mean(((patch - mean_val) / (std(patch) + eps))^3)
    skew_val ← (skew_val + 2) / 4 ▷ Map to [0,1]
    ▷ 2. Frequency Domain Features
    patch_dct ← dct(dct(patch.T, norm='ortho').T, norm='ortho')
    dct_coeffs ← flatten(patch_dct[0:5, 0:5])
    dct_mean ← mean(abs(dct_coeffs))
    dct_mean ← clip(dct_mean / 10.0, 0, 1)
    ▷ 3. Information Theory Features
    hist ← histogram(patch, bins=256, range=(0, 1))
    p ← hist / (sum(hist) + eps)
    entropy ← -sum(p × log2(p + eps)) / 8.0
    ▷ 4. Gradient Features
    gx ← Sobel(patch, CV_32F, 1, 0, ksize=3)
    gy ← Sobel(patch, CV_32F, 0, 1, ksize=3)
    sobel_x ← (mean(gx) + 1) / 2 ▷ Map [-1,1]→[0,1]
    sobel_y ← (mean(gy) + 1) / 2
    features ← [mean_val, std_val, skew_val, dct_mean, entropy, sobel_x, sobel_y]
    return clip(features, 0.0, 1.0)
end function

4.5 Feature Validation System

Algorithm 14 Patch-Strip Correspondence Validation

function FeatureError(feat1, feat2)
    diff ← abs(feat1 - feat2)
    threshold_violations ← [
    diff[0] > 0.03, ▷ Mean intensity
    diff[1] > 0.05, ▷ Standard deviation
    diff[2] > 0.08, ▷ Skewness
    diff[3] > 0.06, ▷ DCT mean
    diff[4] > 0.05, ▷ Entropy
    diff[5] > 0.06, ▷ Sobel X
    diff[6] > 0.06 ▷ Sobel Y
    ]
    return any(threshold_violations)
end function

4.6 Strip Extraction Algorithm

Algorithm 15 Right Image Strip Generation

```
function ExtractStrip(right_image, patch_coords, disparity_range, params)
    y_min, y_max, x_min, x_max ← patch_coords
    strip ← empty((num_disparities, patch_h, patch_w), float16)
    features ← empty((num_disparities, feat_dim), float16)

    for d ← 0 to num_disparities-1 do
        actual_disp ← d × (params.disp_skip + 1) + params.min_disp
        x_start ← x_min - actual_disp
        x_end ← x_max - actual_disp
        strip[d] ← right_image[y_min:y_max, x_start:x_end]
        features[d] ← ComputeFeatures(strip[d])
    end for
    return strip, features
end function
```

5 Module 4: Neural Network Training System

5.1 Module Purpose and Scope

Implements and trains a siamese neural network architecture for disparity estimation using the generated patches and precomputed features. Handles model definition, training pipeline, and evaluation.

5.2 Model Architecture Definition

Algorithm 16 Stereo Matching Model Construction

```
function CreateStereoModel(patch_shape, feature_dim, max_d)
                                                          ▷ Input Definitions
    left_patch_input ← Input(shape=(*patch_shape, 1))
    right_patch_input ← Input(shape=(max_d, *patch_shape, 1))
    left_feat_input ← Input(shape=(feature_dim,))
    right_feat_input ← Input(shape=(max_d, feature_dim))
    mask_input ← Input(shape=(max_d,))
                                                  ▷ Left Image Processing Branch
    x ← Conv2D(32, 3, activation='relu', padding='same')(left_patch_input)
    x ← Conv2D(64, 3, activation='relu', padding='same')(x)
    x ← GlobalAveragePooling2D()(x)
    left_patch_emb ← Dense(128, activation='relu')(x)
                                                      ▷ Left Feature Processing
    left_feat_dense ← Dense(32, activation='relu')(
      LayerNormalization()(left_feat_input))
    E_left ← Concatenate()([left_patch_emb, left_feat_dense])
                                          ▷ Right Image Processing (Time Distributed)
    right_encoder ← BuildPatchEncoder(patch_shape)
    right_patch_emb ← TimeDistributed(right_encoder)(right_patch_input)
                                                     ▷ Right Feature Processing
    right_feat_dense ← TimeDistributed(
      Dense(32, activation='relu'))(LayerNormalization()(right_feat_input))
    E_right ← Concatenate(axis=-1)([right_patch_emb, right_feat_dense])
                                                      ▷ Similarity Computation
    sim_layer ← SimilarityLayer(max_d)
    sim_masked ← sim_layer([E_left, E_right, mask_input])
    out_prob ← Softmax(axis=-1)(sim_masked)
    out_prob ← ClipLayer()(out_prob)                ▷ Ensure numerical stability
    return Model(inputs=[left_patch_input, right_patch_input,
      left_feat_input, right_feat_input, mask_input], outputs=out_prob)
end function
```

5.3 Custom Layer Implementations

5.3.1 Similarity Layer

The Similarity Layer computes cosine similarity between left patch embeddings and right strip embeddings, with masking for invalid disparities.

Algorithm 17 Similarity Layer Implementation

Class: SimilarityLayer
Inherits: Layer
Instance Variables: max_d ▷ Maximum number of disparities
procedure CONSTRUCTOR(max_d)
    SUPER.CONSTRUCTOR
    self.max_d ← max_d
end procedure
procedure CALL(inputs)
    E_left, E_right, mask ← inputs ▷ Unpack three input tensors
    mask ← CAST(mask, bool) ▷ Convert mask to boolean
        ▷ Expand left embedding from (batch, features) to (batch, max_d, features)
    E_left_expanded ← EXPAND_DIMS(E_left, 1) ▷ Add disparity dimension
    E_left_expanded ← TILE(E_left_expanded, [1, self.max_d, 1])
        ▷ Compute cosine similarity
    E_left_normalized ← L2_NORMALIZE(E_left_expanded, axis=-1)
    E_right_normalized ← L2_NORMALIZE(E_right, axis=-1)
    similarity ← REDUCE_SUM(E_left_normalized × E_right_normalized, axis=-1)
        ▷ Apply mask: set invalid disparities to large negative value
    large_negative ← CONSTANT(-1e9)
    similarity_masked ← WHERE(mask, similarity, large_negative)
        ▷ Apply temperature scaling for softer probability distribution
    return similarity_masked / 0.1
end procedure

5.3.2 Clip Layer

The Clip Layer ensures numerical stability by constraining probability values to a valid range, preventing underflow in subsequent operations.

Algorithm 18 Clip Layer Implementation

Class: ClipLayer
Inherits: Layer
procedure CALL(inputs)
        ▷ Clip values to prevent numerical instability in probabilities
    clipped ← CLIP_BY_VALUE(inputs, 1e-7, 1.0)
    return clipped
end procedure

5.4 Data Generator Implementation

The StereoBatchGenerator efficiently loads training data from memory-mapped arrays, handling disparity skipping and proper batching for the neural network.

Algorithm 19 Stereo Batch Generator

Class: StereoBatchGenerator
Inherits: Sequence ▷ Keras Sequence base class
procedure CONSTRUCTOR(parameters)
    self.left_patches ← left_memmap ▷ Memory-mapped left image patches
    self.right_strips ← right_memmap ▷ Memory-mapped right image strips
    self.left_features ← left_feat ▷ Precomputed left patch features
    self.right_features ← right_feat ▷ Precomputed right strip features
    self.mask ← disp_mask ▷ Validity mask for disparities
    self.disparities ← disp_memmap ▷ Ground truth disparities
    self.batch_size ← batch_size
    self.disparity_skip ← disp_skip ▷ Disparity sampling interval
    self.indices ← ARANGE(len(left_memmap)) ▷ All available indices
    self.shuffle ← shuffle
    ON_EPOCH_END ▷ Initial shuffle if enabled
end procedure
function __LEN__
    return CEIL(len(self.indices) / self.batch_size) ▷ Number of batches
end function
procedure __GETITEM__(batch_index)
    ▷ Calculate indices for this batch
    start_idx ← batch_index × self.batch_size
    end_idx ← (batch_index + 1) × self.batch_size
    batch_indices ← self.indices[start_idx:end_idx]
    ▷ Load and preprocess left patches
    left_patches ← self.left_patches[batch_indices]
    left_patches ← EXPAND_DIMS(left_patches, -1) ▷ Add channel dimension
    left_patches ← left_patches.ASTYPE(float32) ▷ Ensure correct dtype
    ▷ Load and preprocess right strips
    right_strips ← self.right_strips[batch_indices]
    right_strips ← EXPAND_DIMS(right_strips, -1)
    right_strips ← right_strips.ASTYPE(float32)
    ▷ Apply disparity skipping if configured
    if self.disparity_skip > 0 **then**
        step ← self.disparity_skip + 1
        right_strips ← right_strips[:, ::step, :, :, :] ▷ Subsample disparities
        strip_features ← self.right_features[batch_indices, ::step, :]
        strip_mask ← self.mask[batch_indices, ::step]
    else
        strip_features ← self.right_features[batch_indices]
        strip_mask ← self.mask[batch_indices]
    end if
    strip_features ← strip_features.ASTYPE(float32)
    patch_features ← self.left_features[batch_indices].ASTYPE(float32)
    ▷ Adjust ground truth disparities for skipping
    disparity_labels ← (self.disparities[batch_indices] // (self.disparity_skip + 1))
    disparity_labels ← disparity_labels.ASTYPE(int32)
    ▷ Package inputs and targets for the model
    model_inputs ← (left_patches, right_strips, patch_features, strip_features, strip_mask)
    return model_inputs, disparity_labels
end procedure
procedure ON_EPOCH_END

5.5 Training Pipeline

Algorithm 20 Complete Training Workflow

```
procedure TrainStereoModel(model, train_gen, val_gen, epochs, lr, k, class_weights)
                                                          ▷ Custom Evaluation Metric
    off_by_k_metric ← CreateOffByKMetric(k)
                                                          ▷ Training Callbacks
    early_stop ← EarlyStopping(monitor='val_metric', patience=20, mode='max')
    checkpoint ← ModelCheckpoint(save_every=5)
                                                          ▷ Model Compilation
    model.compile(
        optimizer=Adam(learning_rate=lr),
        loss='sparse_categorical_crossentropy',
        metrics=[off_by_k_metric]
    )
                                                          ▷ Training Execution
    history ← model.fit(
        train_gen,
        validation_data=val_gen,
        epochs=epochs,
        class_weight=class_weights,
        callbacks=[early_stop, checkpoint]
    )
    return history
end procedure
function CreateOffByKMetric(k)

    function Metric(y_true, y_pred)
        pred_disp ← argmax(y_pred, axis=-1, output_type=int32)
        off_k ← abs(pred_disp - squeeze(y_true)) ≤ k
        return reduce_mean(cast(off_k, float32))
    end function
    return Metric
end function
```

6 System Configuration and Parameters

6.1 Global Configuration

```
# Dataset Parameters
dataset_path = "/mnt/Extra/Project_Storage/stereo_ML_dataset/"
raw_dataset_path = "/mnt/Velocity_Vault/Project_Storage/raw_dataset/"

# Image Processing
resize_fraction = 1.0
resize_factor = 1.0 / resize_fraction
```

```
patch_shape = (16, 16)

# Disparity Range
minimum_disparity = 35
maximum_disparity = 235
disparity_skip = 1
effective_max_disp = (maximum_disparity - minimum_disparity) // (disparity_skip + 1)

# Feature Parameters
feature_dimension = 7

# Training Parameters
batch_size = 32
epochs = 500
learning_rate = 1e-3
training_samples = 2**18  # 262,144
validation_split = 0.1

# Generation Parameters
maximum_tries = 1000000
flush_interval = 10000
```

7 Data Flow and Module Integration

The complete system operates through sequential data transformation:

1. **Raw Web Data → Organized Files**: Data acquisition downloads and structures dataset
2. **Organized Files → Processed Data**: Preprocessing converts to standardized formats
3. **Processed Data → Training Samples**: Patch generation creates feature-rich samples
4. **Training Samples → Trained Model**: Neural network learns disparity estimation

Each module maintains clear interfaces and can operate independently, enabling modular development and testing.